Hao-Ting Ho 何皓廷 · Electrical Engineering, Tunghai University

Estimating the speed of four kinds of robots from an IMU alone (the motion sensor in a phone): one small 2.17M-parameter model

lexho2005@gmail.com GitHub: Jadiouo/tartanimu-unified-io HF: LexHo/tartanimu-a3v20 Kaggle: lexhoooo

PROBLEM AND RESULT

IROS 2026 TartanIMU Challenge: estimate velocity for a car, a quadruped robot dog, a drone and a handheld rig from a 6-axis, 200 Hz IMU alone. **One set of weights, and no label saying which robot it is.** It is hard because the four move very differently, and an IMU does not measure velocity directly, so it must be integrated and errors compound.

20 / 131	**0.19912**	**2.17M**
Kaggle private rank (solo entry)	private score (lower is better; all-zero = 1.0)	parameters; a stronger team's net: 43.7M (about 20x)

20th of 131 on the Kaggle private leaderboard; the organizers subsequently listed our team among the top 10 (organizers' e-mail, 2026-10-09); the official final ranking has not been announced. Official full-test score 0.25878 (organizers' baseline: 0.538). Joined 9/8, submitted 9/21 (about two weeks). In October 2026 the organizers invited me to a working channel for their IMU foundation-model white paper; I am not a listed author and it is unpublished.

APPROACH

- **Small network**: the signal is cut into 1-second windows, read by a 1-D convolutional network and linked across 40 windows by a GRU (a sequence network); two small modules first correct sensor bias and orientation, and a final step carries velocity from window to window.
- **Self-supervised pretraining**: hide parts of the IMU signal and reconstruct them, using train and val IMU only.
- **Physics-consistent augmentation**: replay recordings faster and correct the accelerometer (at least 4 teams used the same family independently, so it is not novel, and it changed together with longer pretraining); and a second one that makes drone-racing data faster in a drag-consistent way. A counterfactual probe showed the model ignored drag, which motivated it. Its four target sequences are all in the public half, so it did not affect the private rank; its official gain (0.0044) is within random-seed variation.
- **Evaluation**: the official validation set cannot see the hardest fast flights, so I built two extra exams and used run-to-run variation from random seeds (standard deviation about 0.0056) as the threshold. The same recipe with another seed scores 0.26933; 0.25878 is not a stable value.

AFTER THE CONTEST (UNRANKED, SEPARATE FROM THE CONTEST RESULT)

Another team (thisray) released a physics-based post-processing method (code under Apache-2.0) after the challenge. I put it on top of the same network (**Step 1**), then added two steps of my own: **Step 2, a correction for ground robots**, and **Step 3, per-window uncertainty-weighted smoothing**. The method cuts drone velocity error by 49.0% on held-out recordings but barely helps ground robots (car velocity error +14.6%), so they get half the correction.

- On a held-out set of 27 ground recordings the model never saw, Step 2 improves the score by about 5.7% to 6.1% in three training runs, intervals excluding 0. The 0.5 factor was chosen after the fact and the three checks share the same recordings.
- Kaggle returned for Step 3: public 0.22342 / private 0.13903; approximate full-test score about 0.178 (conversion error about ±0.0005).
- Step 1 (about 0.182) and thisray's reported 0.188 are within noise; I cannot say which is better.
- Disclosure: at least 6 post-contest submissions used the returned scores for selection. I also made about 89 one-sequence probe submissions to recover which test sequences are in the public or private half; the values revealed were not used to adjust any output, and I have not yet asked the organizers whether the method may be described publicly.

MY ROLE

I was the **research lead**: I made the final calls on research direction, trade-offs, validation rules, and whether each submission went out. Implementation, experiment runs, coding and cross-review were done by AI coding agents (Claude Code, Codex).

Ideas that were mine:

- Different physical models per platform (flight dynamics for the drone)
- Feeding per-window uncertainty into the smoother (this became Step 3)
- Trying multi-alignment averaging (predicting the same data under several alignments and averaging): no effect, not adopted
- Pushing the score further after the contest, and deciding which post-processing steps were worth submitting

METHOD: GUARDING AGAINST AI SELF-VERIFICATION

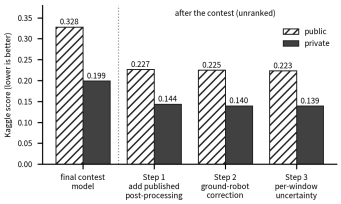
- **Held-out data**: 39 ground recordings kept out of training and used as the exam (the pretraining still saw their IMU, so not strict).
- **Pass/fail rule written before each submission**: 12 files; self-archived, no third-party timestamp, some in the same code snapshot (commit) as the results.
- **Recomputation by another model plus red-team review** (an independent model attacks the result): Codex re-ran the ground analysis (difference <1e-6); review also caught our own mistakes.
- **Replication with other seeds**: Step 2 holds on two more seeds.
- **Negative results recorded**: test-time multi-view averaging failed; dropping the car segment was not significant; a network that predicts its own uncertainty was weaker than using disagreement between predictions, so it was not submitted.
- **Leakage audit**: an external dataset overlapping the test set was dropped.
- **Disclosure**: choices between candidates, 33 official queries, post-contest test feedback.

SKILLS

- Form testable hypotheses and check them with counterfactual experiments
- Design evaluation, with noise-based thresholds, not just tune models
- End-to-end pipeline: data, training, cloud (Kaggle, GPUtw), reproducible delivery (re-run on CPU in an isolated environment in 85 s)
- Direct multi-agent research; credit and licence others' work explicitly

NEXT STEP

I have tried most of the directions I can think of; the next step is to deploy the model on a physical robot and make it a system that works in practice.



Kaggle-returned public / private scores (exact). Post-contest submissions are unranked, and their basis differs from the contest result, so this is not one progress line.

All scores: lower is better. Contest result: Kaggle private 0.19912 (20th of 131). Post-contest figures are late submissions and carry no rank. thisray's numbers are from their technical report and were not re-verified here. · superodometry.com/imuchallenge